

BACKEND ENGINEERING

Sriniously Backend Engineering Guide

Complete Visual Reference for Modern Backend Development

ENHANCED EDITION

Table of Contents

01 HTTP Fundamentals

02 Routing

03 Serialization

04 Authentication & Authorization

05 Validation & Transformation

06 Backend Architecture

07 REST API Design

08 Databases & Postgres

09 Caching

10 Background Tasks

11 Search & Elasticsearch

12 Error Handling

13 Configuration Management

14 Logging, Monitoring & Observability

15 Graceful Shutdown

16 Security

17 Scaling & Performance Part 1

18 Scaling & Performance Part 2

01 — HTTP Fundamentals

HTTP is the universal language of the web. Every API call, every page load, every mobile app talking to a server — it all runs on HTTP. Understanding HTTP deeply means you can debug mysterious failures, design faster APIs, and make security decisions with confidence. It is the foundation everything else in backend engineering rests on.

Core Principles

HTTP is a **stateless, request-response protocol**. The client sends a request; the server sends back a response. The server does not remember previous requests unless you explicitly add state (sessions, tokens). HTTP is also **text-based** (mostly), making it human-readable and easy to debug with tools like curl or browser DevTools.

Stateless

Each request is independent. The server has no memory of past interactions unless you add it.

Request-Response

Client always initiates. Server waits, processes, and replies. No server push (without WebSockets/SSE).

Layered

HTTP sits on top of TCP. HTTPS adds TLS encryption on top of that stack.

HTTP Versions

Version	Key Feature	When to Use
HTTP/1.1	Persistent connections, chunked transfer	Legacy systems, simple servers
HTTP/2	Multiplexing, header compression, binary framing	Default for modern web — faster page loads
HTTP/3	Built on QUIC (UDP), faster connection setup	High-latency networks, mobile apps

HTTP Message Structure

Every HTTP message has three parts: a **start line**, **headers**, and an optional **body**.

```
GET /api/users/42 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbG...


```

← Request line (method, path, version)
← Headers (metadata)
← Blank line separates headers from body
← Body (empty for GET requests)

HTTP Methods

Method	Purpose	Idempotent?	Safe?
GET	Retrieve a resource	Yes	Yes
POST	Create a resource or trigger action	No	No
PUT	Replace entire resource	Yes	No
PATCH	Partial update	No	No

Method	Purpose	Idempotent?	Safe?
DELETE	Remove a resource	Yes	No
HEAD	Like GET but no body (check existence)	Yes	Yes
OPTIONS	Discover allowed methods (CORS preflight)	Yes	Yes

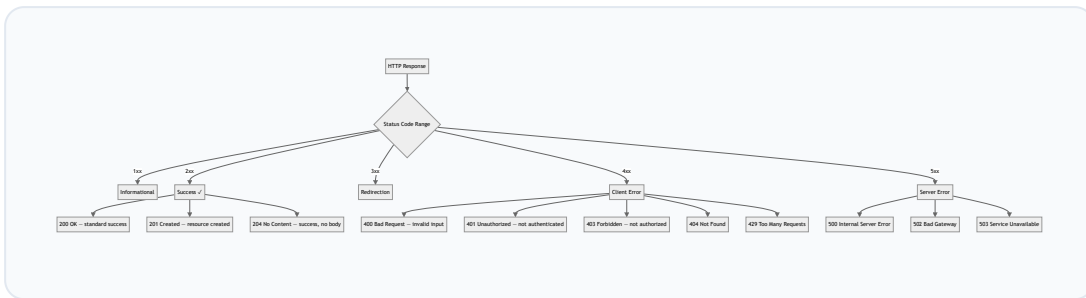
NOTE

Safe methods never modify server state. **Idempotent** methods produce the same result if called multiple times — critical for retry logic.

Important Headers

Header	Direction	Purpose
Content-Type	Both	Format of the body (e.g., application/json)
Authorization	Request	Credentials (Bearer token, Basic auth)
Accept	Request	What response format the client wants
Cache-Control	Both	Caching directives (max-age, no-cache)
ETag	Response	Resource version fingerprint for conditional requests
Set-Cookie	Response	Store data in the client's browser
Location	Response	Redirect URL (301/302/201 responses)

Status Codes



CORS (Cross-Origin Resource Sharing)

Browsers block JavaScript from making requests to a different domain than the page origin — this is the **Same-Origin Policy**. CORS is the mechanism that lets servers explicitly allow cross-origin requests.

```
// Server must respond with these headers for cross-origin requests:
Access-Control-Allow-Origin: https://myapp.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
```

WARNING

Never use `Access-Control-Allow-Origin: *` with credentials. It is a security vulnerability. Always specify exact origins.

Caching

HTTP caching reduces server load and speeds up responses. The client or an intermediary (CDN, proxy) stores responses and reuses them.

- **Cache-Control: max-age=3600** — cache for 1 hour
- **ETag + If-None-Match** — conditional request; server returns 304 if unchanged
- **Cache-Control: no-store** — never cache (use for sensitive data)
- **Cache-Control: private** — only browser caches, not CDN

Compression

Large responses waste bandwidth. Enable gzip or Brotli compression on your server. A JSON response can shrink 70–90%. The client sends `Accept-Encoding: gzip, br` and the server responds with `Content-Encoding: gzip`.

Large File Transfers

- **Chunked Transfer Encoding** — stream data in chunks without knowing total size upfront
- **Range Requests** — client requests bytes 0–1023 with `Range: bytes=0-1023`; enables resume downloads
- **Multipart Upload** — split large files into parts, upload separately, combine server-side (S3-style)

HTTPS

HTTPS = HTTP + TLS encryption. It protects data in transit from eavesdropping and tampering. Always use HTTPS in production. Let's Encrypt provides free certificates. HTTPS also enables HTTP/2 and is required for modern browser features.

KEY TAKEAWAY

HTTP is the contract between client and server. Master methods, status codes, headers, and caching — and you can debug any API issue. Always use HTTPS, understand idempotency for retries, and configure CORS explicitly rather than allowing everything.

02 — Routing

Routing is how your server decides which code runs for each incoming URL. A well-designed routing layer makes your API predictable, versionable, and easy to maintain. Poor routing leads to spaghetti endpoints, broken links, and confused frontend developers.

Static vs Dynamic Routes

Static Routes

Fixed paths like `/health` or `/api/users`. Same handler every time.

Dynamic Routes

Paths with parameters like `/api/users/:id`. The `:id` segment is extracted and passed to your handler.

```
// Express.js examples
app.get('/api/users', listUsers);           // Static
app.get('/api/users/:id', getUser);         // Dynamic — req.params.id
app.get('/api/users/:id/posts/:postId', getPost); // Multiple params
```

Path Parameters vs Query Parameters

Type	Location	Use For	Example
Path params	URL path	Identifying a specific resource	<code>/users/42</code>
Query params	After <code>?</code>	Filtering, sorting, pagination	<code>/users?role=admin&page=2</code>

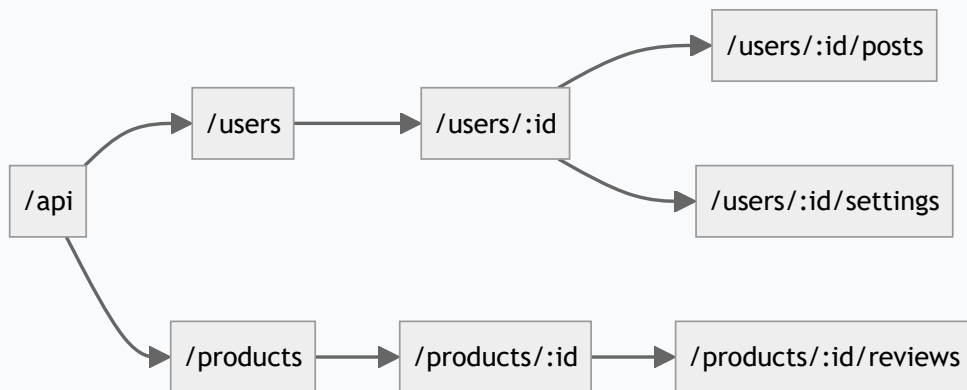
TIP

Use path params for *what* resource (identity) and query params for *how* to present it (filters, pagination). GET `/users/42` not GET `/users?id=42` .

Nested Routes

Express parent-child relationships in the URL hierarchy:

GET <code>/users/42/posts</code>	→ All posts by user 42
GET <code>/users/42/posts/7</code>	→ Post 7 by user 42
POST <code>/users/42/posts</code>	→ Create post for user 42



API Versioning

When you need to make breaking changes without breaking existing clients:

- **URL versioning** — `/api/v1/users` , `/api/v2/users` (most common, clearest)
- **Header versioning** — `Accept: application/vnd.myapp.v2+json`
- **Query param** — `/api/users?version=2` (least recommended)

Catch-All Routes

Match any remaining path segments. Useful for SPA fallbacks or wildcard proxies:

```
app.get('/docs/*', serveDocs);    // Matches /docs/anything/here
app.get('*', serveSPA);           // Fallback for client-side routing
```

KEY TAKEAWAY

Design routes as a resource hierarchy. Use path params for identity, query params for options. Version your API from day one, and keep routes RESTful and predictable — your frontend team will thank you.

03 — Serialization

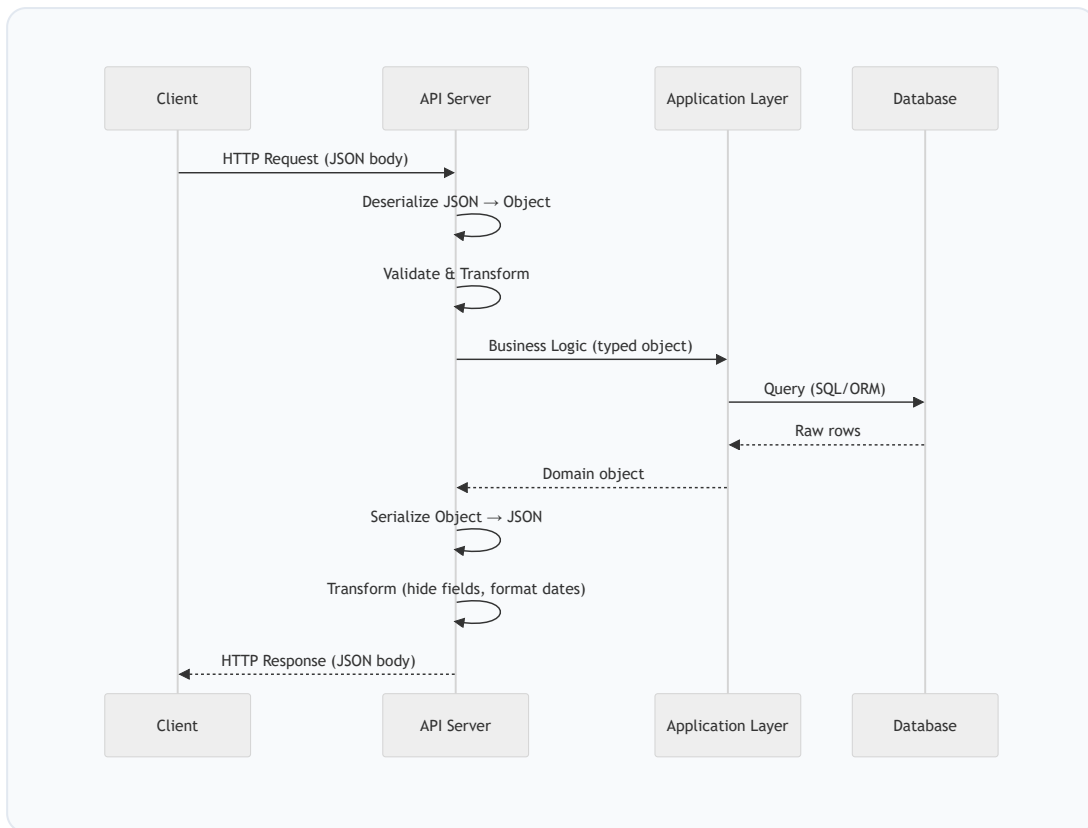
Serialization is converting data between your application's internal format (objects, structs) and a wire format (JSON, XML) for transmission. Every API you build serializes on the way out and deserializes on the way in. Getting this wrong causes data loss, security bugs, and broken clients.

JSON — The Universal Format

JSON (JavaScript Object Notation) is the de facto standard for REST APIs. It is human-readable, lightweight, and supported by every language.

```
{
  "id": 42,
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "roles": ["admin", "editor"],
  "profile": {
    "avatar": "https://cdn.example.com/a.jpg",
    "bio": "Backend engineer"
  },
  "createdAt": "2024-01-15T10:30:00Z"
}
```

Serialization Workflow



Common Serialization Patterns

- **DTOs (Data Transfer Objects)** — purpose-built objects for API responses, separate from database models
- **Field exclusion** — never serialize passwords, internal IDs, or secrets
- **Date formatting** — always use ISO 8601 (`2024-01-15T10:30:00Z`)
- **Null handling** — decide policy: omit null fields or include them explicitly
- **Enums** — serialize as strings (`"active"`) not integers (`1`) for readability

TIP

Never expose your database schema directly. Use DTOs/response models to control exactly what leaves your server. This prevents accidental data leaks when you add sensitive columns later.

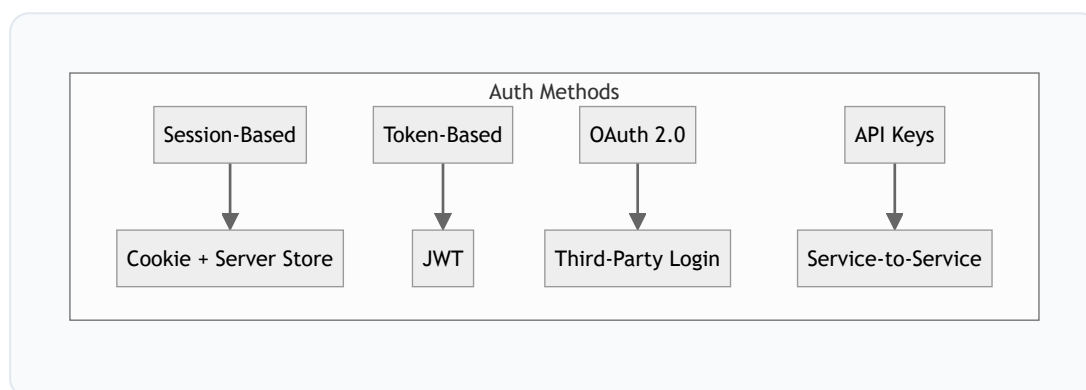
KEY TAKEAWAY

Serialization is your API's public contract. Use JSON, ISO 8601 dates, string enums, and DTOs to decouple your internal models from what clients see. Validate on the way in; transform on the way out.

04 — Authentication & Authorization

Authentication answers "Who are you?" Authorization answers "What are you allowed to do?" These are the gatekeepers of every secure backend. Confusing them or implementing them poorly is one of the most common sources of security breaches.

Authentication Methods



Session-Based Authentication

The server creates a session after login and stores it (in memory, Redis, or database). A session ID is sent to the client as an **HttpOnly cookie**.

1. `POST /login { email, password }`
2. Server validates credentials
3. Server creates session: `{ userId: 42, expires: ... }`
4. Server responds: `Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict`
5. Client sends cookie automatically on every request
6. Server looks up session → identifies user

JWT (JSON Web Tokens)

Stateless tokens containing encoded user claims. The server verifies the signature without a database lookup.

```
// JWT structure: header.payload.signature
eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiI0MiIsImV4cCI6MTcwNTMyNDAwMH0.signature

// Send in Authorization header:
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

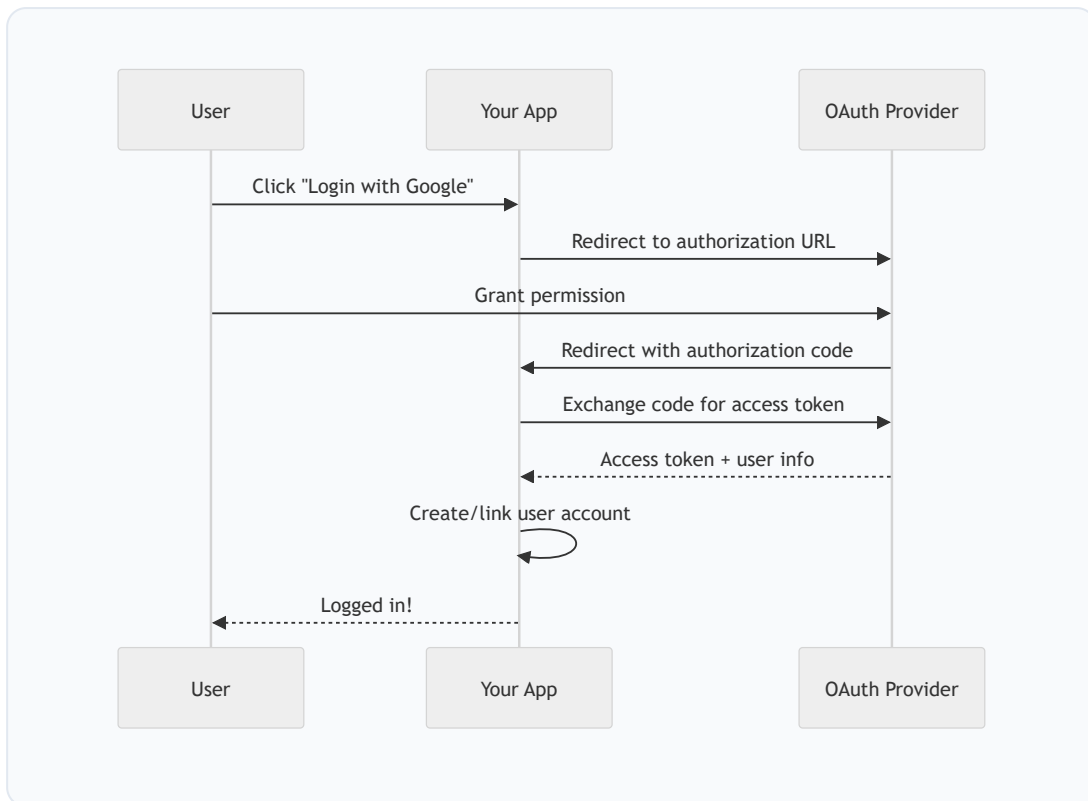
Feature	Sessions	JWT
State	Stateful (server stores session)	Stateless (self-contained)
Revocation	Easy — delete session	Hard — need blocklist or short expiry
Scalability	Needs shared session store	Scales horizontally easily
Storage	HttpOnly cookie (secure)	Memory/localStorage (XSS risk)
Best for	Traditional web apps	APIs, microservices, mobile

Cookies — Security Flags

- **HttpOnly** — JavaScript cannot access (prevents XSS token theft)
- **Secure** — only sent over HTTPS
- **SameSite=Strict** — not sent on cross-site requests (prevents CSRF)
- **Max-Age / Expires** — automatic expiration

OAuth 2.0

Lets users log in via third-party providers (Google, GitHub) without sharing passwords with your app.



Authorization — RBAC

Role-Based Access Control assigns permissions to roles, and roles to users.

```
User "Alice" → Role "Admin" → Permissions [read, write, delete, manage_users]
User "Bob"   → Role "Editor" → Permissions [read, write]
User "Carol" → Role "Viewer" → Permissions [read]
```

Security Best Practices

- Hash passwords with **bcrypt** or **argon2** — never store plaintext
- Use short JWT expiry (15 min access + refresh token pattern)
- Implement rate limiting on login endpoints
- Enable MFA for sensitive operations
- Validate tokens on every request — fail closed (deny by default)
- Never put sensitive data in JWT payloads (they are base64, not encrypted)

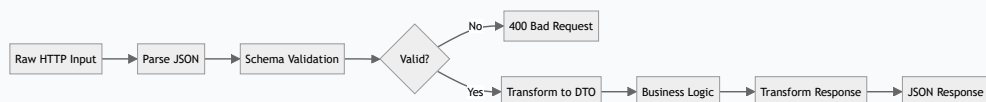
KEY TAKEAWAY

Authenticate identity, authorize actions. Use sessions for web apps, JWTs for APIs. Always hash passwords, set secure cookie flags, and implement RBAC. When in doubt, deny access — fail closed, not open.

05 — Validation & Transformation

Never trust client input. Validation is your first line of defense against bad data, injection attacks, and confused users. Transformation converts raw input into clean, typed objects your business logic can safely work with. Skip validation and you will ship bugs and security holes.

The Validation Pipeline



Validation Types

Structural

Is it the right shape? Required fields present? Correct types?

Format

Valid email? Phone number? UUID? Date format?

Business Rules

Is username available? Is stock sufficient? Age > 18?

Sanitization

Trim whitespace, strip HTML, normalize unicode

```
// Example with Zod (TypeScript)
const CreateUserSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8).max(128),
  age: z.number().int().min(13),
```

```
role: z.enum(['user', 'admin']).default('user'),
});
```

Frontend vs Backend Validation

Aspect	Frontend	Backend
Purpose	User experience (instant feedback)	Security and data integrity
Can be bypassed?	Yes — always	No — this is the real gate
What to validate	Format, required fields	Everything + business rules
Rule	Nice to have	Mandatory — never skip

WARNING

Frontend validation is for UX only. A malicious user can send any HTTP request directly to your API, bypassing your UI entirely. Backend validation is non-negotiable.

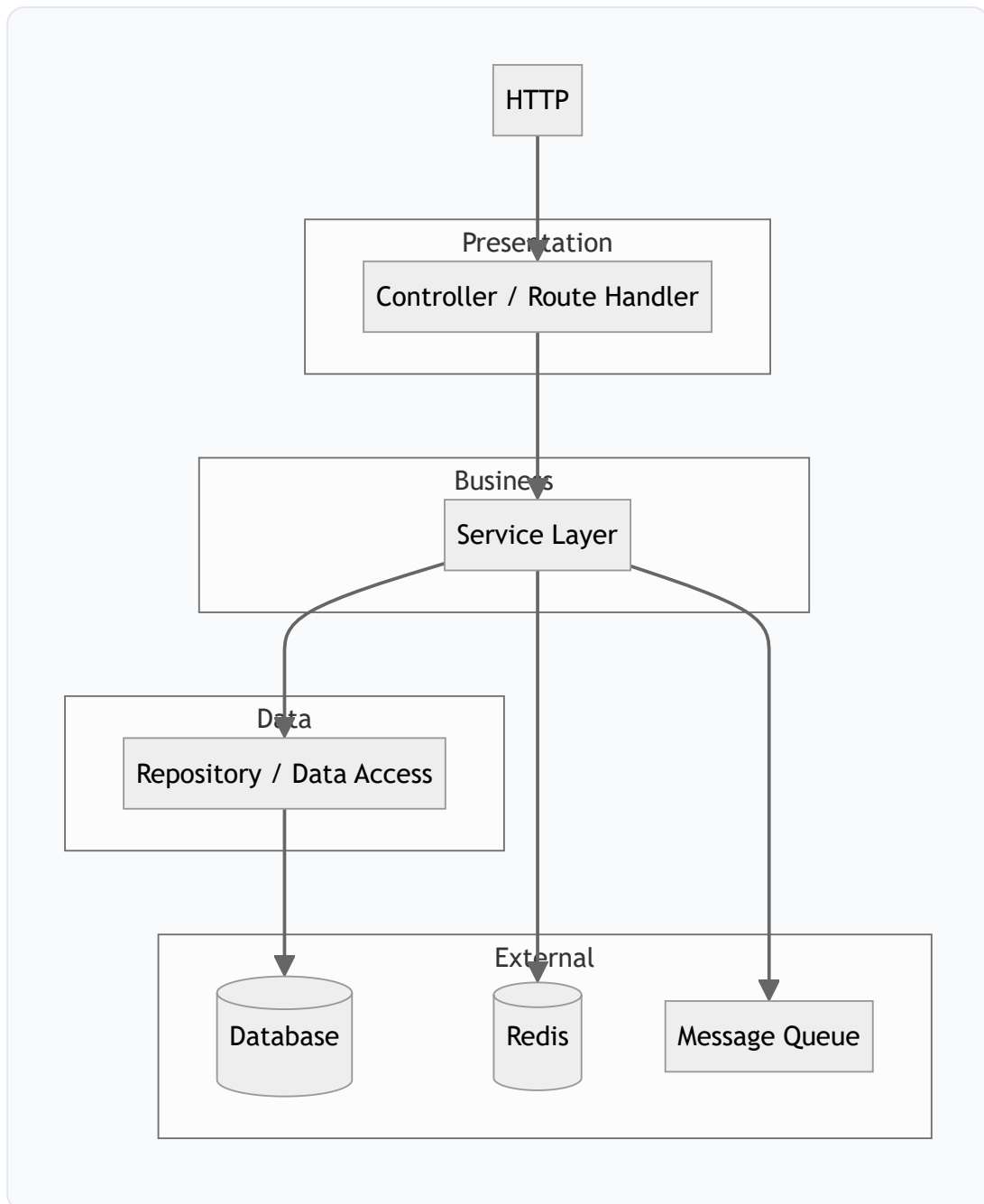
KEY TAKEAWAY

Validate early, validate everything on the backend. Use schema libraries (Zod, Joi, Pydantic) for structural validation, then apply business rules in your service layer. Transform data at the boundaries — raw input in, clean DTOs out.

06 — Backend Architecture

Good architecture separates concerns so each part of your codebase has one job. When a bug appears or requirements change, you know exactly where to look. The Controller → Service → Repository pattern is the most widely used backend architecture for good reason — it scales from small projects to large teams.

Layered Architecture



Layer Responsibilities

Layer	Responsibility	Should NOT
Controller	Parse request, call service, return HTTP response	Contain business logic or SQL queries
Service	Business rules, orchestration, transactions	Know about HTTP headers or status codes

Layer	Responsibility	Should NOT
Repository	Database queries, data mapping	Make business decisions

```
// Controller – thin, HTTP-aware
async function createUser(req, res) {
  const dto = CreateUserSchema.parse(req.body);
  const user = await userService.createUser(dto);
  res.status(201).json(toUserResponse(user));
}

// Service – business logic
async function createUser(dto) {
  if (await userRepo.findByEmail(dto.email)) throw new ConflictError('Email taken');
  const hash = await bcrypt.hash(dto.password, 12);
  return userRepo.create({ ...dto, password: hash });
}

// Repository – data access only
async function create(data) {
  return db.query('INSERT INTO users ...', data);
}
```

Middleware

Middleware functions run between receiving a request and sending a response. They form a pipeline:

```
Request → [Logger] → [CORS] → [Auth] → [RateLimit] → [Router] → Response
           ↑           ↑           ↑           ↑           ↑
        logs req   sets headers verifies JWT/token throttles requests route handler
```

Request Context

Pass request-scoped data (authenticated user, request ID, locale) through the call chain without global variables:

- **AsyncLocalStorage** (Node.js) — implicit context propagation
- **Context object** — explicit parameter passed to service methods
- **Dependency injection** — inject request-scoped services per request

KEY TAKEAWAY

Keep controllers thin, services focused on business logic, and repositories data-only. Use middleware for cross-cutting concerns (auth, logging, CORS). This separation makes your codebase testable, maintainable, and team-friendly.

07 — REST API Design

REST (Representational State Transfer) is an architectural style for designing networked APIs. Well-designed REST APIs are intuitive — developers can guess your endpoints. Poor REST leads to inconsistent, hard-to-document APIs that frustrate every consumer.

The 6 REST Constraints

1. Client-Server

Separate concerns — UI and API evolve independently.

2. Stateless

Each request contains all info needed. No server-side session state.

3. Cacheable

Responses must define themselves as cacheable or not.

4. Uniform Interface

Consistent URLs, standard methods, self-descriptive messages.

5. Layered System

Client cannot tell if connected to end server or intermediary.

6. Code on Demand

Optional — server can send executable code (rarely used).

Resource-Oriented Routes

GET	/users	→ List all users
GET	/users/42	→ Get user 42
POST	/users	→ Create a user
PUT	/users/42	→ Replace user 42 entirely
PATCH	/users/42	→ Partially update user 42


```
DELETE /users/42      → Delete user 42

GET    /users/42/orders → List orders for user 42
POST   /users/42/orders → Create order for user 42
```

Idempotency

Method	Idempotent?	Safe to Retry?
GET, PUT, DELETE	Yes	Yes — same result every time
POST	No	Risk of duplicates — use idempotency keys
PATCH	No	Depends on implementation

TIP

For non-idempotent POST requests (payments, orders), accept an `Idempotency-Key` header. Store the key and return the same response if the client retries.

List API Best Practices

```
GET /users?page=2&limit=20&sort=-createdAt&filter[role]=admin
```

Response:

```
{
  "data": [...],
  "meta": {
    "page": 2,
    "limit": 20,
    "total": 156,
    "totalPages": 8
  },
  "links": {
```

```
"self": "/users?page=2",
"next": "/users?page=3",
"prev": "/users?page=1"
}
}
```

Status Code Usage

Scenario	Code
Successful GET	200 OK
Resource created	201 Created (+ Location header)
Successful DELETE	204 No Content
Validation error	400 Bad Request
Not logged in	401 Unauthorized
Logged in but not allowed	403 Forbidden
Resource not found	404 Not Found
Conflict (duplicate email)	409 Conflict
Rate limited	429 Too Many Requests

KEY TAKEAWAY

Design APIs around resources (nouns), not actions (verbs). Use HTTP methods correctly, paginate list endpoints, return proper status codes, and use idempotency keys for critical POST operations. Consistency beats cleverness.

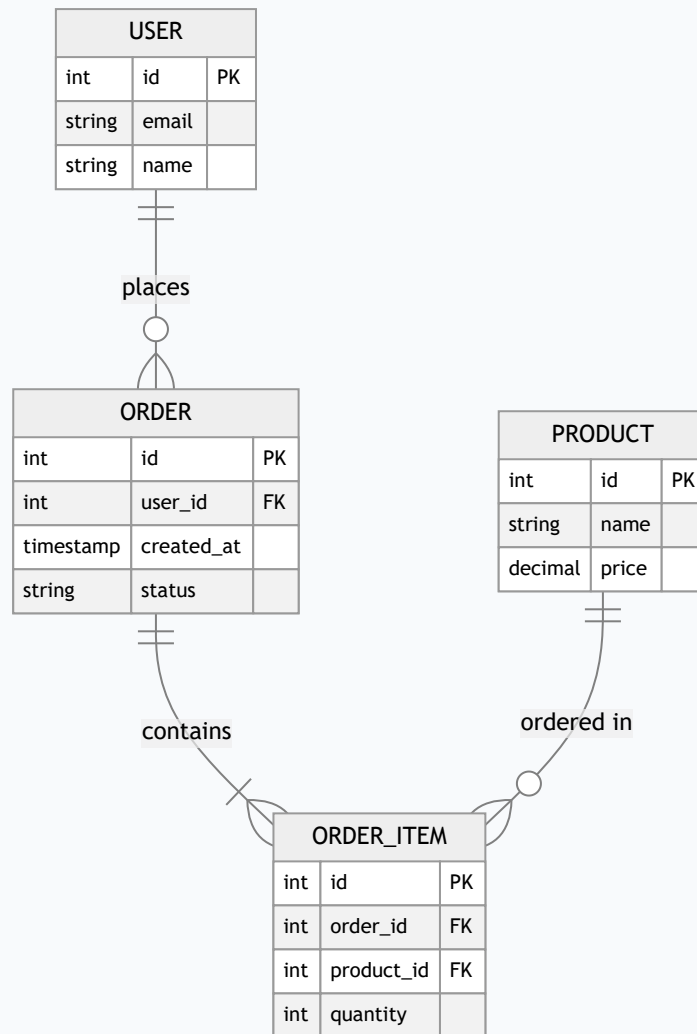
08 — Databases & PostgreSQL

Your database is the source of truth for all persistent data. PostgreSQL is the most capable open-source relational database — it handles everything from simple CRUD to complex analytics, JSON documents, and full-text search. Understanding databases deeply separates junior from senior backend engineers.

PostgreSQL Data Types

Category	Types	Use For
Numeric	<code>INTEGER</code> , <code>BIGINT</code> , <code>DECIMAL</code> , <code>SERIAL</code>	IDs, counts, money (use <code>DECIMAL</code> for money)
Text	<code>VARCHAR(n)</code> , <code>TEXT</code> , <code>CHAR</code>	Names, descriptions (<code>TEXT</code> for unlimited)
Boolean	<code>BOOLEAN</code>	Flags (<code>is_active</code> , <code>is_verified</code>)
Date/Time	<code>TIMESTAMP</code> , <code>DATE</code> , <code>INTERVAL</code>	Always store UTC timestamps
JSON	<code>JSONB</code>	Flexible schemas, metadata, settings
UUID	<code>UUID</code>	Public-facing IDs (non-sequential, secure)
Array	<code>TEXT[]</code> , <code>INTEGER[]</code>	Tags, simple lists

Relationships



- **One-to-Many** — User has many Orders (`user_id` FK on orders table)
- **Many-to-Many** — Students and Courses via junction table `enrollments`
- **One-to-One** — User and Profile (share same PK or FK with UNIQUE constraint)

Constraints

- **PRIMARY KEY** — unique identifier for each row
- **FOREIGN KEY** — enforces referential integrity between tables
- **UNIQUE** — no duplicate values (e.g., email)
- **NOT NULL** — field must have a value
- **CHECK** — custom validation (e.g., `price > 0`)

- **DEFAULT** — automatic value if none provided

Indexes

Indexes speed up reads but slow down writes. Add them on columns you frequently filter or join on.

```
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_created_at ON orders(created_at DESC);

-- Composite index for common query patterns
CREATE INDEX idx_orders_user_status ON orders(user_id, status);
```

TIP

Index foreign keys, columns in WHERE clauses, and columns in ORDER BY. Use `EXPLAIN ANALYZE` to see if your queries use indexes. Avoid over-indexing — each index adds write overhead.

Migrations

Version-controlled database schema changes. Never modify production databases manually.

```
-- Migration: 001_create_users.sql
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Migration: 002_add_user_role.sql
ALTER TABLE users ADD COLUMN role VARCHAR(50) DEFAULT 'user';
```

SQL Injection Prevention

DANGER

Never concatenate user input into SQL strings. Always use parameterized queries or an ORM.

```
// ❌ VULNERABLE – SQL injection
db.query(`SELECT * FROM users WHERE email = '${email}'`);

// ✅ SAFE – parameterized query
db.query('SELECT * FROM users WHERE email = $1', [email]);
```

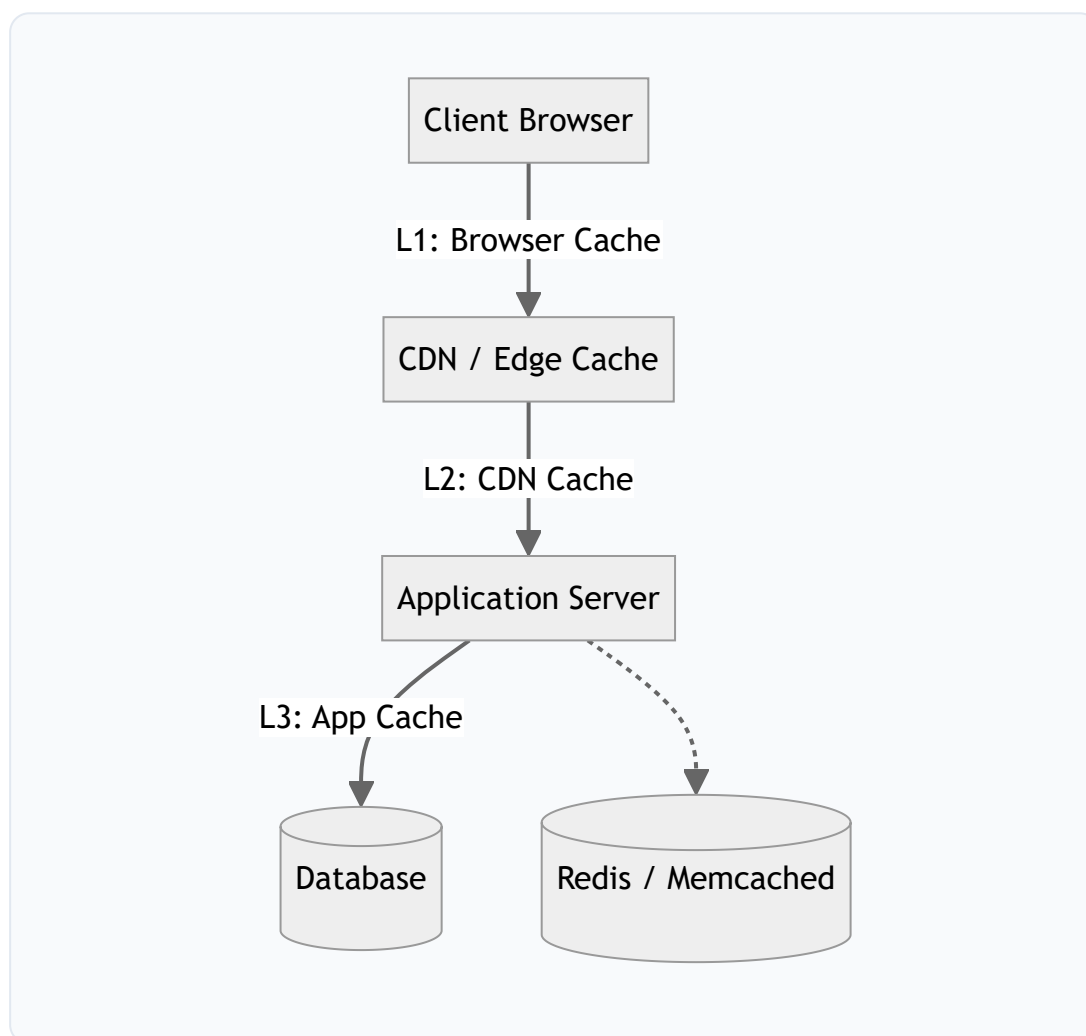
KEY TAKEAWAY

PostgreSQL is your source of truth — design schemas with proper types, constraints, and relationships. Index strategically, migrate version-controlled, and always use parameterized queries. Run EXPLAIN ANALYZE on slow queries before adding indexes blindly.

09 — Caching

Caching stores frequently accessed data in fast storage so you do not recompute or re-fetch it every time. A well-placed cache can turn a 500ms database query into a 2ms memory lookup — a 250x improvement. But caching adds complexity: stale data, invalidation bugs, and memory pressure. Use it strategically.

Three Levels of Caching



Level	Location	Speed	Best For
L1 — Browser	Client device	Instant	Static assets, Cache-Control

Level	Location	Speed	Best For
			headers
L2 — CDN	Edge servers globally	~10ms	Static files, public API responses
L3 — Application	Redis/Memcached in your stack	~1ms	Database query results, session data

Caching Strategies

Cache-Aside

App checks cache first. On miss, reads DB, writes to cache. Most common pattern.

Write-Through

Write to cache and DB simultaneously. Cache always fresh but writes are slower.

Write-Behind

Write to cache immediately, async flush to DB. Fast writes but risk of data loss.

Read-Through

Cache automatically loads from DB on miss. App only talks to cache.

Cache-Aside Pattern (Most Common)

```

async function getUser(id) {
  const cacheKey = `user:${id}`;
  let user = await redis.get(cacheKey);
  if (user) return JSON.parse(user);    // Cache HIT

  user = await db.query('SELECT * FROM users WHERE id = $1', [id]); // Cache
MISS
  if (user) await redis.setex(cacheKey, 3600, JSON.stringify(user)); // TTL:
1 hour

```



```
return user;
}
```

Eviction Policies

Policy	Behavior	When to Use
TTL (Time-To-Live)	Expire after fixed duration	Most common — set per key
LRU (Least Recently Used)	Evict oldest accessed item when full	Memory-bound caches
LFU (Least Frequently Used)	Evict least accessed item	When access patterns are stable
Explicit Invalidation	Delete cache key on data update	When freshness is critical

Common Use Cases

- **Database query results** — cache expensive JOINS and aggregations
- **Session data** — fast user session lookups in Redis
- **API responses** — cache public, slow-changing endpoints
- **Rate limiting counters** — track request counts per IP/user
- **Computed values** — leaderboard scores, analytics dashboards

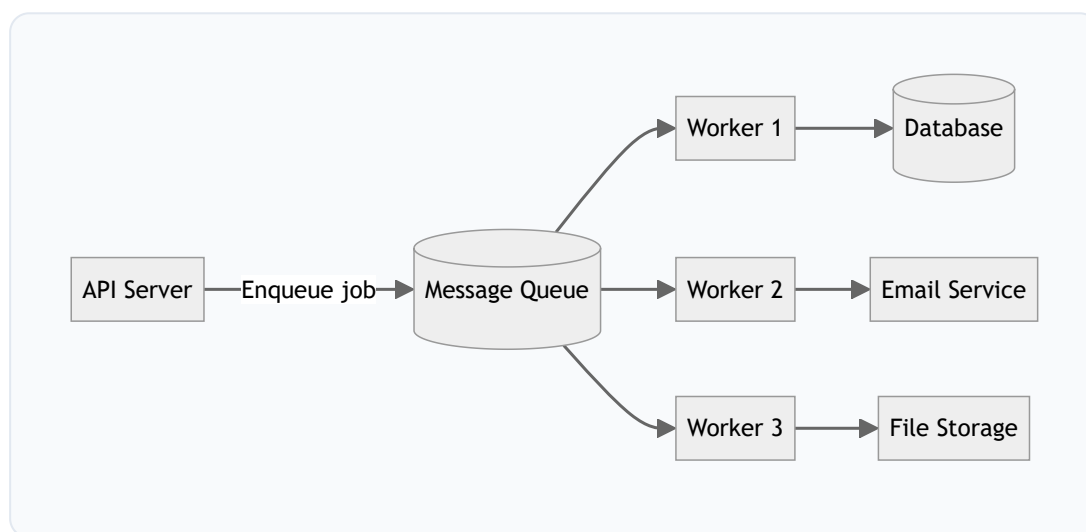
KEY TAKEAWAY

Cache at the right level for the right data. Use cache-aside with TTLs for most cases. Always plan cache invalidation — stale data bugs are subtle and painful. Monitor hit rates to know if your cache is effective.

10 — Background Tasks

Not everything should happen during an HTTP request. Sending emails, processing images, generating reports — these slow operations should run in the background so your API responds instantly. Background task systems are essential for any production backend.

Queue Architecture



Types of Background Tasks

Type	Description	Examples
Fire-and-Forget	Enqueue and move on, no result needed	Send welcome email, log analytics event
Scheduled	Run at specific time or interval	Daily report, cleanup old sessions
Delayed	Run after a delay	Retry failed payment in 5 minutes

Type	Description	Examples
Recurring	Run on cron schedule	Sync inventory every hour
Priority	High-priority jobs processed first	Password reset before newsletter

Popular Tools

Bull / BullMQ

Redis-backed queues for Node.js. Great DX, retries, priorities.

Celery

Python task queue. Mature, supports RabbitMQ and Redis brokers.

Sidekiq

Ruby background processing with Redis. Battle-tested at scale.

AWS SQS

Managed message queue. No infrastructure to manage.

Best Practices

- **Make jobs idempotent** — safe to retry without side effects
- **Set retry limits** — with exponential backoff (1s, 2s, 4s, 8s...)
- **Dead letter queue** — move permanently failed jobs for manual inspection
- **Monitor queue depth** — growing queue = workers cannot keep up
- **Keep jobs small** — break large tasks into smaller chunks
- **Return job ID** — let clients poll status for long-running tasks

```
// API responds immediately with job ID
POST /reports → 202 Accepted { "jobId": "abc-123", "status": "processing" }
```

```
GET /reports/abc-123/status → { "status": "completed", "url": "/reports/abc-123.pdf" }
```

KEY TAKEAWAY

Offload slow work to background queues. Respond fast to users, process asynchronously. Make every job idempotent, set retry policies, monitor queue health, and always provide a way to check job status.

11 — Search & Elasticsearch

Database `LIKE` queries do not scale for real search. When users need to find products, articles, or users by keyword with relevance ranking, fuzzy matching, and faceted filters — you need a dedicated search engine. Elasticsearch is the industry standard.

Why Not Database Search?

Feature	SQL LIKE	Elasticsearch
Speed at scale	Slow (full table scan)	Milliseconds on billions of docs
Relevance ranking	None	BM25 scoring algorithm
Typo tolerance	None	Fuzzy matching built-in
Faceted search	Complex GROUP BY	Native aggregations
Autocomplete	Very difficult	Edge n-grams, completion suggester

Inverted Index

Elasticsearch builds an inverted index — a map from each word to the documents containing it. This is how it finds matches in milliseconds.

Documents:

```
Doc1: "quick brown fox"
Doc2: "lazy brown dog"
Doc3: "quick red fox"
```

Inverted Index:

```
"quick" → [Doc1]
"brown" → [Doc1, Doc2]
"fox" → [Doc1, Doc3]
"lazy" → [Doc2]
```

"dog"	→ [Doc2]
"red"	→ [Doc3]

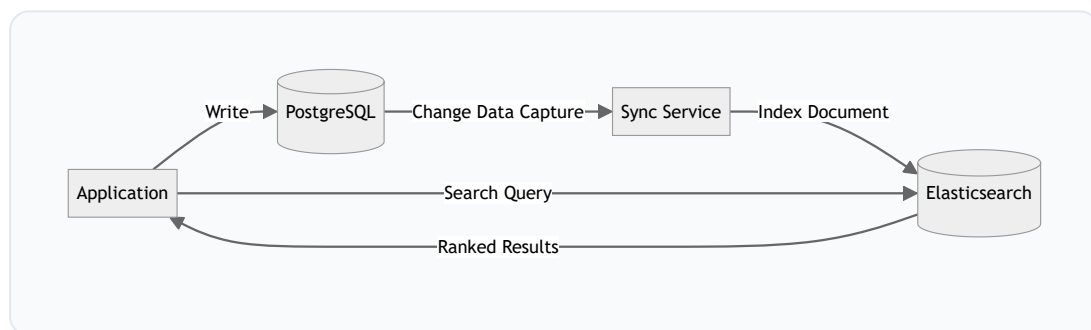
Query "quick fox" → Doc1 (both terms), Doc3 (one term)

BM25 Scoring

BM25 ranks documents by relevance. It considers:

- **Term frequency** — how often the term appears in the document
- **Inverse document frequency** — rare terms score higher than common ones
- **Document length** — shorter matching documents rank slightly higher

Architecture Pattern



Common Use Cases

- **Product search** — e-commerce with filters (price, category, brand)
- **Log analysis** — ELK stack (Elasticsearch, Logstash, Kibana)
- **Autocomplete** — search-as-you-type suggestions
- **Analytics** — aggregations over large datasets
- **Geo search** — find restaurants within 5km

KEY TAKEAWAY

Use Elasticsearch when you need fast, relevant, full-text search at scale. Keep PostgreSQL as source of truth and sync to Elasticsearch.

Understand inverted indexes and BM25 to tune search relevance effectively.

12 — Error Handling

Errors will happen — networks fail, databases timeout, users send bad input. How your backend handles errors determines whether debugging takes 5 minutes or 5 hours. Consistent, secure error handling is a hallmark of production-quality code.

Five Types of Errors

1. Validation Errors

Bad input from client. Return 400 with field-level details.

2. Authentication Errors

Missing or invalid credentials. Return 401.

3. Authorization Errors

Valid user, insufficient permissions. Return 403.

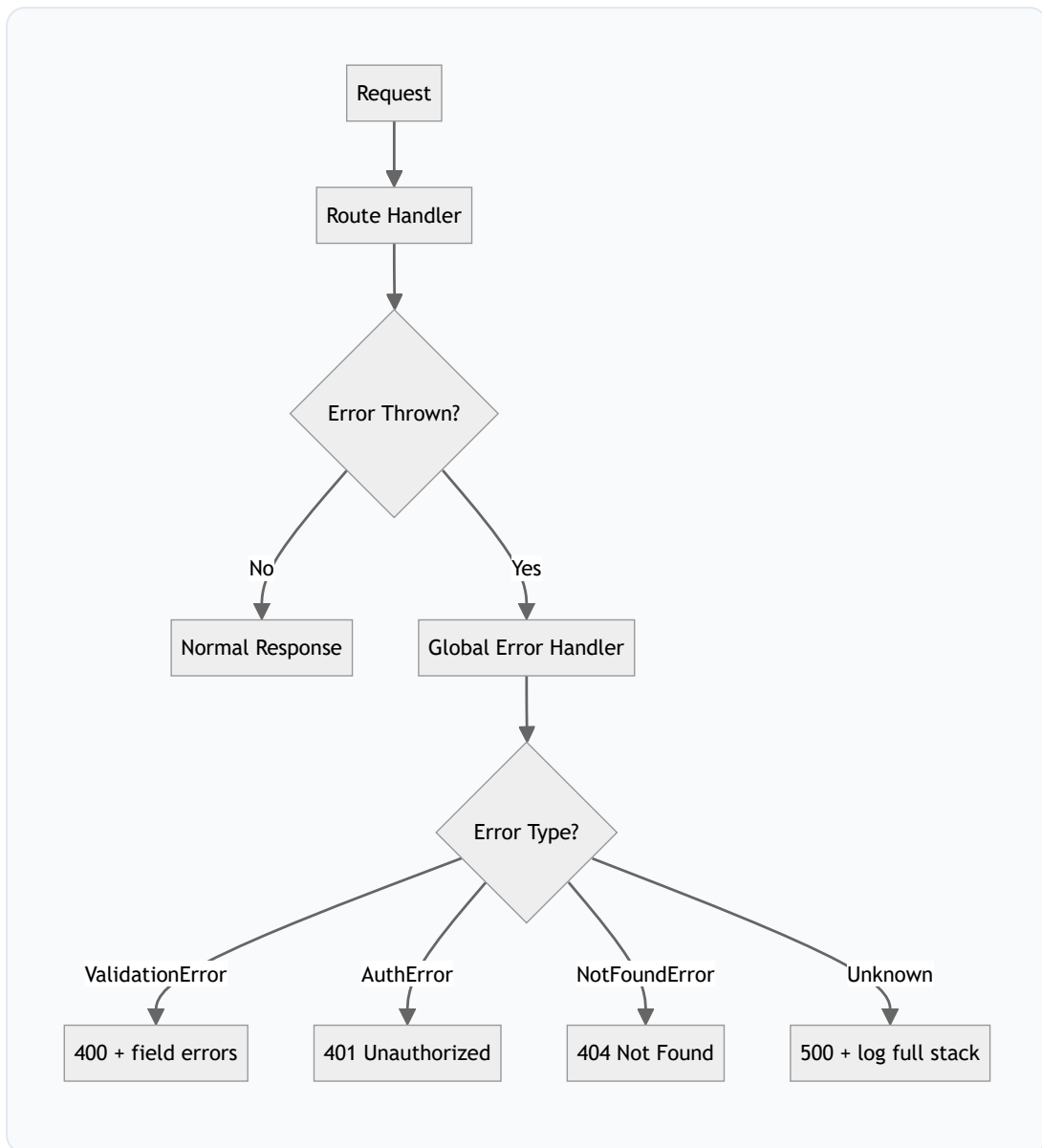
4. Not Found Errors

Resource does not exist. Return 404.

5. Server Errors

Unexpected internal failures. Return 500. Log details, hide from client.

Global Error Handler



Consistent Error Response Format

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": [
      { "field": "email", "message": "Must be a valid email address" },
      { "field": "password", "message": "Must be at least 8 characters" }
    ]
  },
  "requestId": "req_abc123"
}
```

Security in Error Handling

- **Never expose stack traces** to clients in production
- **Never reveal** whether an email exists in login errors ("Invalid credentials" not "Email not found")
- **Log everything** server-side with request ID, user ID, timestamp
- **Include request ID** in error responses so users can report issues
- **Do not leak** database errors, file paths, or internal IDs

DANGER

Returning `500 Internal Server Error: relation "users" does not exist at character 15` tells attackers your database schema. Always sanitize error messages in production.

KEY TAKEAWAY

Categorize errors, handle them centrally, and respond consistently. Return helpful messages to clients, log detailed info server-side. Never leak internal details — security and debuggability must coexist.

13 — Configuration Management

Hardcoded configuration is a ticking time bomb. Database URLs, API keys, feature flags, and port numbers change between environments and over time. Proper configuration management lets you deploy the same code to development, staging, and production — just with different settings.

Five Types of Configuration

1. Environment Config

Settings that differ per environment: DB URLs, log levels, debug mode.

2. Secrets

API keys, passwords, certificates. Never in code or git.

3. Feature Flags

Toggle features without deploying. Enable beta features for select users.

4. Application Config

Business settings: pagination limits, upload max size, rate limits.

5. Infrastructure Config

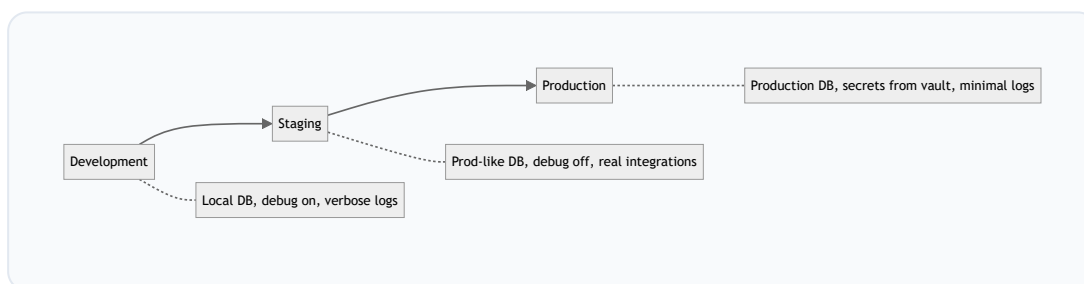
Port bindings, worker counts, connection pool sizes.

Configuration Storage Options

Method	Best For	Example
Environment Variables	Secrets, env-specific values	<code>DATABASE_URL=postgres://...</code>

Method	Best For	Example
.env files (local only)	Local development convenience	<code>.env.development</code>
Config files (YAML/JSON)	Non-secret app settings	<code>config/production.yaml</code>
Secret managers	Production secrets	AWS Secrets Manager, Vault, Doppler
Feature flag services	Dynamic toggles	LaunchDarkly, Unleash, Flagsmith

Environment Strategy



Golden Rules

- **12-Factor App** — store config in the environment, not in code
- **Never commit secrets** — use `.gitignore` for `.env` files
- **Validate config at startup** — fail fast if required vars are missing
- **Defaults for non-critical values** — but require explicit values for secrets
- **Same code, different config** — build once, deploy everywhere
- **Document all variables** — maintain a `.env.example` file

```

# .env.example (committed to git)
DATABASE_URL=postgres://user:pass@localhost:5432/myapp
REDIS_URL=redis://localhost:6379
JWT_SECRET=change-me-in-production
  
```

```
PORT=3000  
LOG_LEVEL=info
```

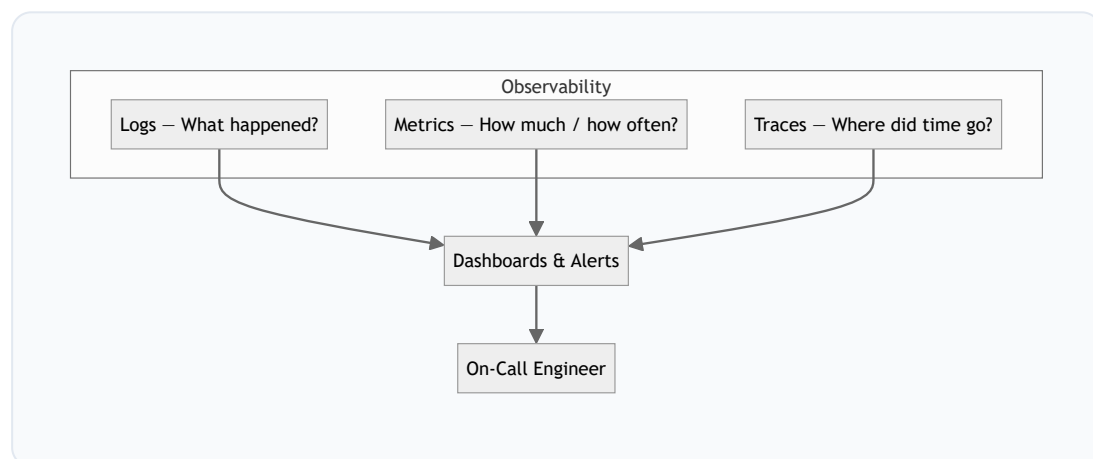
KEY TAKEAWAY

Separate config from code. Use environment variables for secrets, validate at startup, and never commit credentials. The same artifact should run in every environment — only configuration changes.

14 — Logging, Monitoring & Observability

You cannot fix what you cannot see. When your API is slow at 3 AM or users report errors, observability is how you find the problem. The three pillars — logs, metrics, and traces — give you complete visibility into your system's behavior.

The Three Pillars



Pillar	Data Type	Tools	Answers
Logs	Discrete events with context	ELK, Loki, CloudWatch	"What happened at 3:04 AM?"
Metrics	Numeric measurements over time	Prometheus, Grafana, Datadog	"Is error rate increasing?"
Traces	Request journey across services	Jaeger, Zipkin, OpenTelemetry	"Which service is slow?"

Structured Logging

Always log in structured JSON format — not plain strings. This enables powerful querying and filtering.

```
// ❌ Unstructured – hard to search
console.log("User 42 logged in from 192.168.1.1");

// ✅ Structured – queryable
logger.info({
  event: "user.login",
  userId: 42,
  ip: "192.168.1.1",
  requestId: "req_abc123",
  duration: 45
});
```

Key Metrics to Track

- **Request rate** — requests per second (throughput)
- **Error rate** — percentage of 5xx responses
- **Latency** — p50, p95, p99 response times
- **Saturation** — CPU, memory, connection pool usage
- **Business metrics** — signups, orders, active users

Observability Workflow



TIP

Include a unique `requestId` in every log entry, trace span, and error response. This single ID connects logs, metrics, and traces for any given request.

KEY TAKEAWAY

Instrument everything from day one. Use structured logs, track the four golden signals (latency, traffic, errors, saturation), and correlate with request IDs. Observability is not optional in production — it is how you sleep at night.

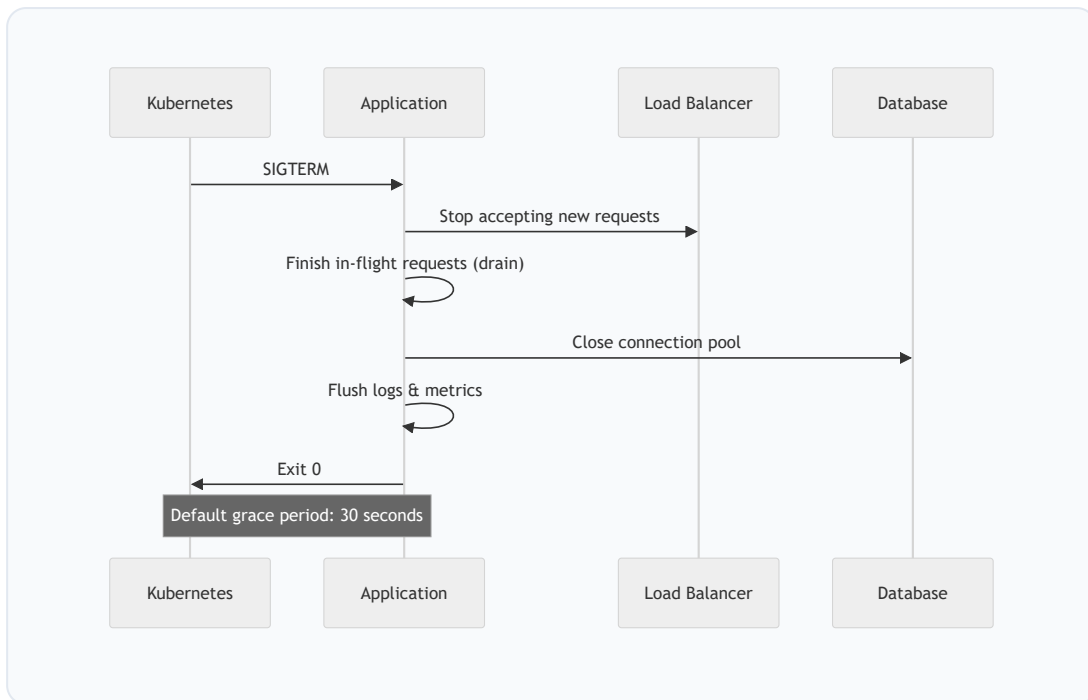
15 — Graceful Shutdown

When you deploy new code or scale down servers, processes receive termination signals. A graceful shutdown finishes in-flight requests, closes database connections, and flushes logs before exiting. Without it, users get 502 errors and data can be corrupted mid-write.

Shutdown Signals

Signal	Source	Behavior
<code>SIGTERM</code>	Kubernetes, Docker, systemd	Polite "please shut down" — handle this
<code>SIGINT</code>	Ctrl+C in terminal	Interactive interrupt — handle this too
<code>SIGKILL</code>	<code>kill -9</code> , OOM killer	Immediate death — cannot be caught

Graceful Shutdown Sequence



Connection Draining

Stop accepting new connections but let existing requests complete:

```
const server = app.listen(PORT);

process.on('SIGTERM', async () => {
  console.log('SIGTERM received. Starting graceful shutdown...');
  server.close(async () => { // Stop accepting new connections
    await db.pool.end(); // Close database connections
    await redis.quit(); // Close Redis connection
    await logger.flush(); // Flush remaining logs
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 30000); // Force exit after 30s
});
```

Cleanup Checklist

- Stop accepting new HTTP connections
- Wait for in-flight requests to complete (with timeout)
- Close database connection pools
- Disconnect from Redis, message queues

- Flush buffered logs and metrics
- Complete or fail in-progress background jobs cleanly

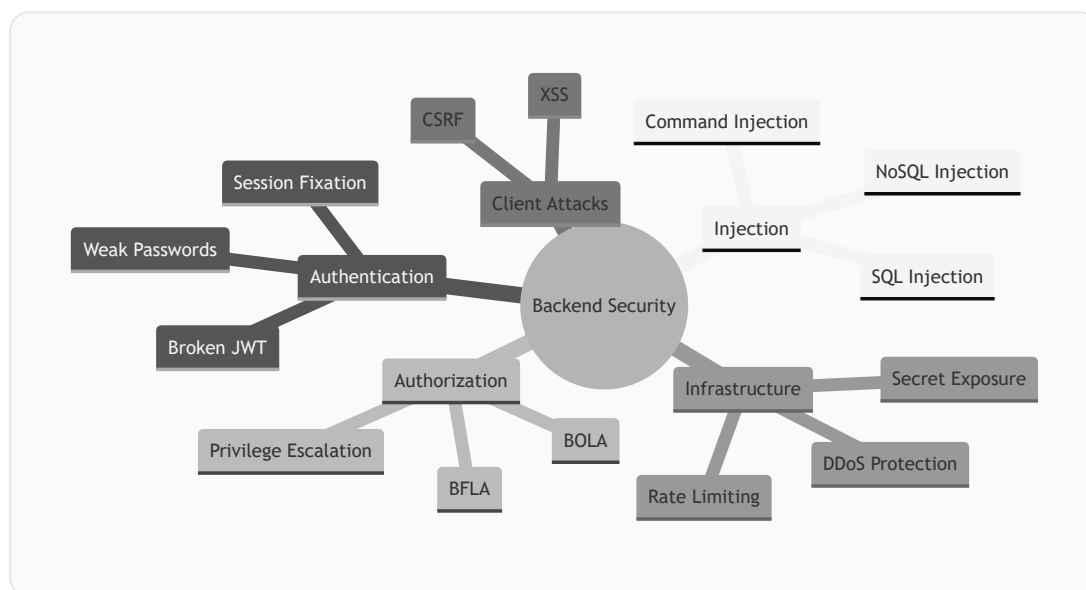
KEY TAKEAWAY

Always handle SIGTERM and SIGINT. Drain connections, close resources, then exit. In Kubernetes, you have ~30 seconds before SIGKILL — make every second count. Zero-downtime deploys depend on graceful shutdown.

16 — Security

Security is not a feature you add at the end — it is a property of every layer of your system. A single vulnerability can expose all user data, destroy trust, and end your business. Backend engineers are the last line of defense.

Security Threat Map



Injection Attacks

The most common and dangerous vulnerability class. Never trust user input.

Attack	Vector	Prevention
SQL Injection	User input in SQL queries	Parameterized queries, ORMs
Command Injection	User input in shell commands	Never pass input to exec/spawn
NoSQL Injection	User input in MongoDB queries	Validate and sanitize input types

Password Security

- Hash with **bcrypt** (cost factor 12+) or **argon2id**
- Never store plaintext passwords — not even temporarily
- Enforce minimum length (8+) but do not impose complexity rules that reduce entropy
- Use timing-safe comparison for token/password checks
- Implement account lockout after failed attempts

Rate Limiting

```
// Token bucket: 100 requests per minute per IP
Rate Limit Rules:
- Login endpoint:    5 requests / minute / IP
- API endpoints:    100 requests / minute / user
- Password reset:   3 requests / hour / email
- File upload:      10 requests / hour / user
```

BOLA & BFLA

BOLA — Broken Object Level Authorization

User accesses another user's resource by changing an ID. `GET /orders/999` when order 999 belongs to someone else. Fix: always verify ownership in your service layer.

BFLA — Broken Function Level Authorization

Regular user accesses admin endpoints. `DELETE /admin/users/42` as a non-admin. Fix: check roles/permissions on every endpoint, not just the UI.

XSS & CSRF

Attack	What It Does	Backend Defense
XSS (Cross-Site Scripting)	Injects malicious JS into pages	Sanitize output, set Content-Security-Policy header, HttpOnly cookies
CSRF (Cross-Site Request Forgery)	Tricks browser into making authenticated requests	SameSite cookies, CSRF tokens, verify Origin/Referer headers

Security Checklist

- HTTPS everywhere — no exceptions
- Parameterized queries — no string concatenation in SQL
- Input validation on every endpoint
- Authentication and authorization on every protected route
- Rate limiting on auth and sensitive endpoints
- Secrets in environment variables or vault — never in code
- Dependency scanning — keep packages updated
- Security headers — HSTS, CSP, X-Frame-Options, X-Content-Type-Options

KEY TAKEAWAY

Security is layered defense. Validate all input, parameterize all queries, authorize every action, rate-limit sensitive endpoints, and hash all passwords. Assume every request is malicious until proven otherwise.

17 — Scaling & Performance (Part 1)

Performance is a feature. Users abandon apps that feel slow, and infrastructure costs scale with inefficiency. Before you add more servers, understand where time is actually spent. Measure first, optimize the biggest bottleneck, then measure again.

Key Performance Metrics

Latency

Time to respond. Track p50, p95, p99 — averages hide outliers.

Throughput

Requests per second your system handles.

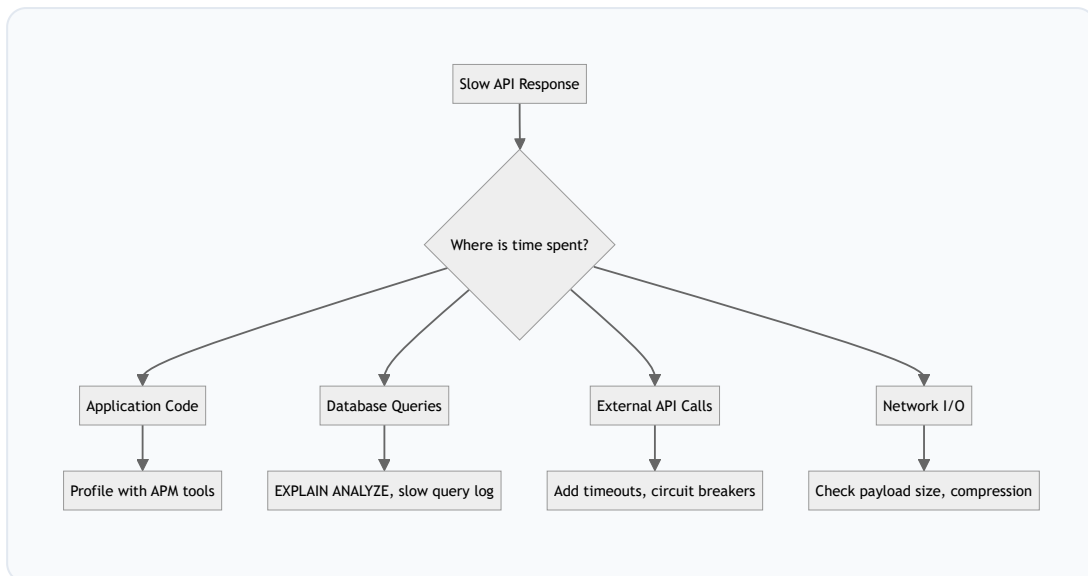
Error Rate

Percentage of failed requests. Even 0.1% matters at scale.

Saturation

How full your resources are — CPU, memory, connections, disk I/O.

Finding Bottlenecks



Database Optimization

- **Index strategically** — columns in WHERE, JOIN, ORDER BY clauses
- **Avoid N+1 queries** — use JOINS or eager loading instead of loops
- **Connection pooling** — reuse DB connections (PgBouncer, built-in pools)
- **Query only what you need** — select specific columns, use pagination
- **Read replicas** — offload read-heavy queries to replica databases
- **Denormalize carefully** — duplicate data to avoid expensive JOINS when reads dominate

```
-- ❌ N+1 Problem: 1 query for users + N queries for orders
const users = await db.query('SELECT * FROM users LIMIT 10');
for (const user of users) {
  user.orders = await db.query('SELECT * FROM orders WHERE user_id = $1',
    [user.id]);
}

-- ✅ Fixed: 1 query with JOIN or 2 queries with IN clause
const users = await db.query(`
  SELECT u.*, o.id as order_id, o.total
  FROM users u LEFT JOIN orders o ON o.user_id = u.id
  LIMIT 10
`);
```

Caching Patterns for Performance

Pattern	What to Cache	TTL
Query result cache	Expensive DB query results	5–60 minutes
Object cache	Frequently read entities (user profiles)	15–60 minutes
Page cache	Rendered API responses	1–5 minutes
CDN cache	Static assets, public API responses	Hours to days

Vertical vs Horizontal Scaling

Aspect	Vertical (Scale Up)	Horizontal (Scale Out)
Method	Bigger server (more CPU/RAM)	More servers behind load balancer
Limit	Hardware ceiling	Near-unlimited
Complexity	Simple — no code changes	Requires stateless design
Cost	Expensive at high end	Linear, commodity hardware
When	Quick fix, small apps	Production systems at scale

TIP

Scale vertically first (easier), then horizontally when you hit hardware limits. But design for horizontal scaling from the start — stateless services, external session stores, connection pooling.

KEY TAKEAWAY

Measure before optimizing. Fix N+1 queries, add indexes, cache hot data, and pool connections. Vertical scaling is a quick win; horizontal scaling is the long-term strategy. Profile your code — intuition about bottlenecks is usually wrong.

18 — Scaling & Performance (Part 2)

Once you have optimized a single server, scaling out means designing systems where multiple servers work together seamlessly. This section covers the architecture patterns that power the world's largest applications — from load balancers to microservices to serverless.

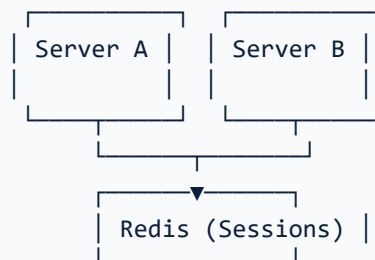
Stateless Services

Any server can handle any request. No session data stored in memory. This is the prerequisite for horizontal scaling.

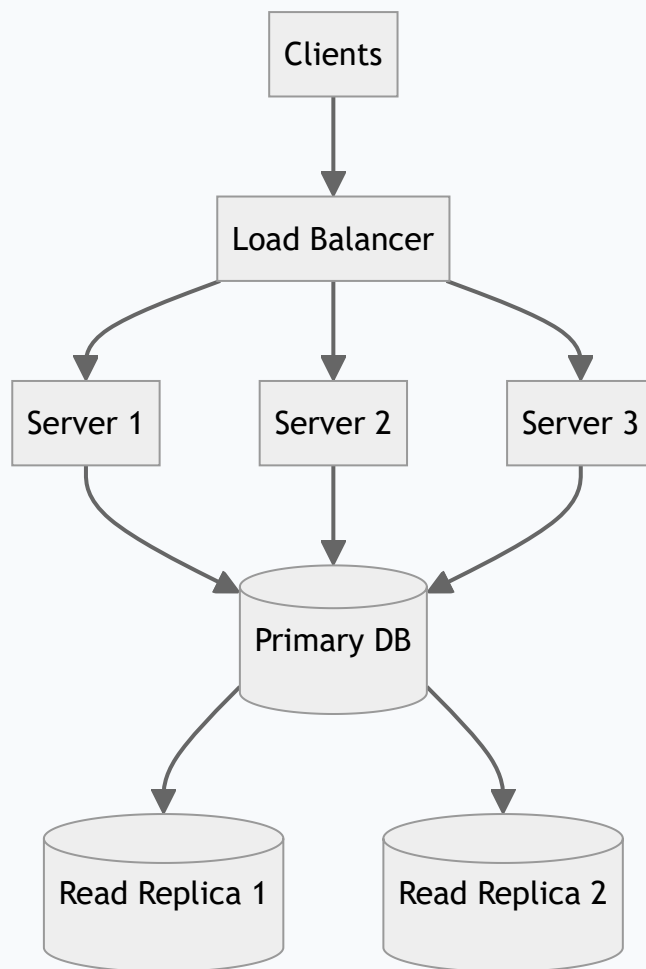
Stateful (Bad for scaling):



Stateless (Good for scaling):



Load Balancers



Algorithm	Behavior	Best For
Round Robin	Rotate through servers equally	Homogeneous servers, equal requests
Least Connections	Send to server with fewest active connections	Variable request durations
IP Hash	Same client always goes to same server	Session affinity (avoid if possible)
Weighted	More powerful servers get more traffic	Mixed hardware

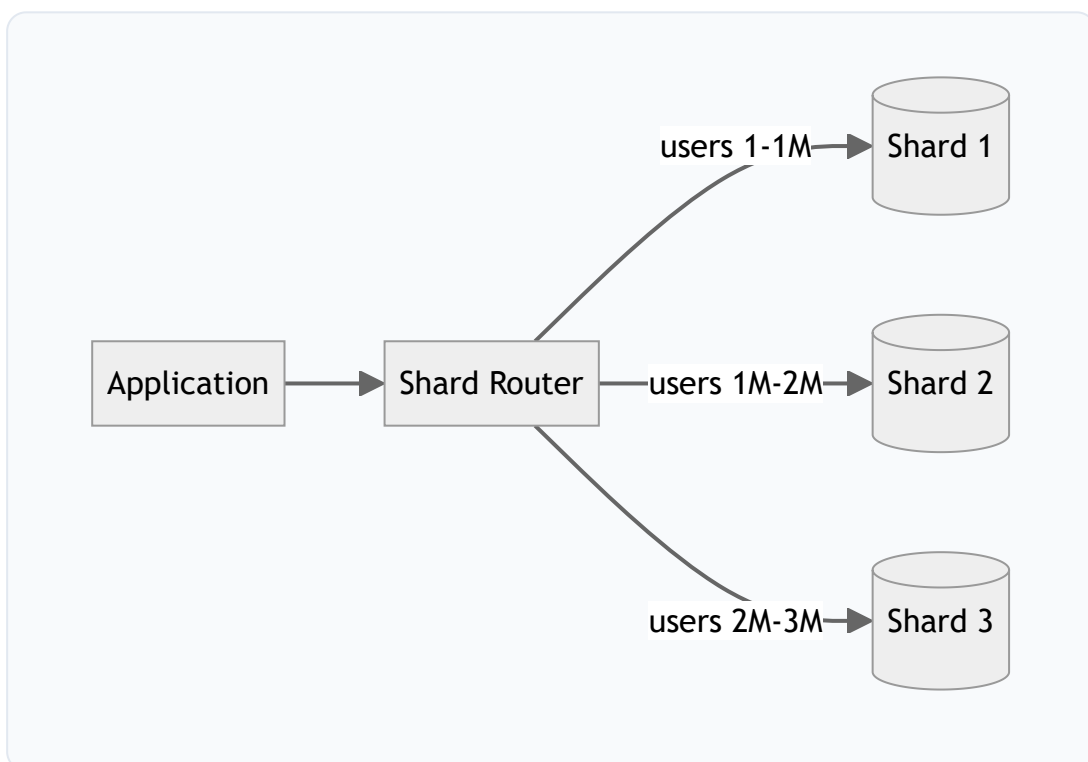
Read Replicas

Separate read and write database operations. Primary handles writes; replicas handle reads.

- **Write** → Primary database only
- **Read** → Any replica (eventually consistent)
- Replication lag is typically milliseconds — acceptable for most reads
- Do not read your own writes on replicas for critical flows (use primary)

Database Sharding

Split data across multiple databases when a single server cannot hold all data or handle all queries.



Choose shard key carefully — user ID is common. Avoid cross-shard queries; they defeat the purpose.

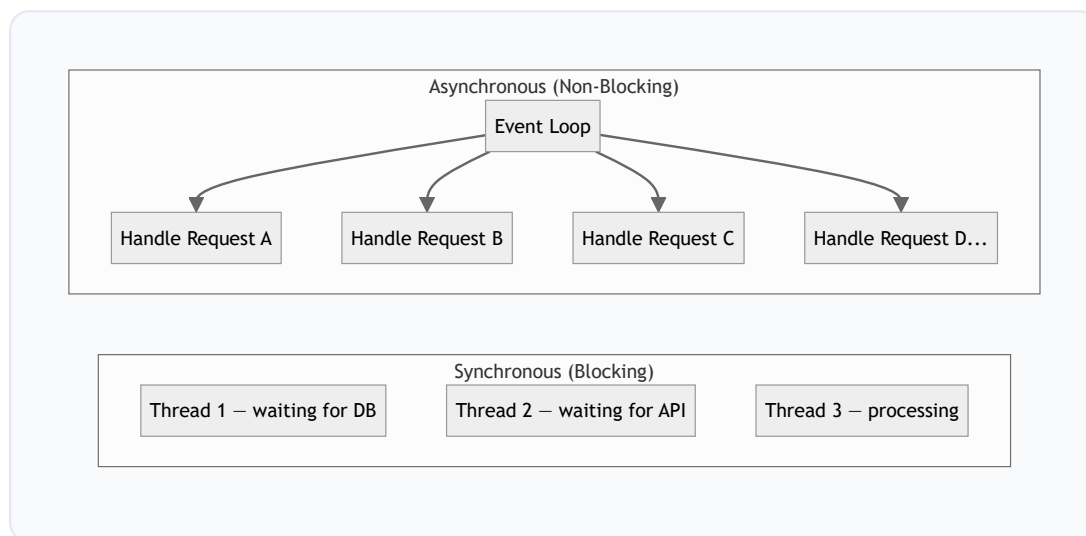
CDN (Content Delivery Network)

Cache static assets and API responses at edge locations worldwide. Users get content from the nearest server.

- Static files: images, CSS, JS, fonts — cache for days/weeks
- API responses: public, cacheable endpoints — cache for minutes
- Reduces origin server load by 80–95% for cacheable content
- Providers: Cloudflare, AWS CloudFront, Fastly

Async & Non-Blocking I/O

Do not block threads waiting for I/O. Handle thousands of concurrent connections efficiently.

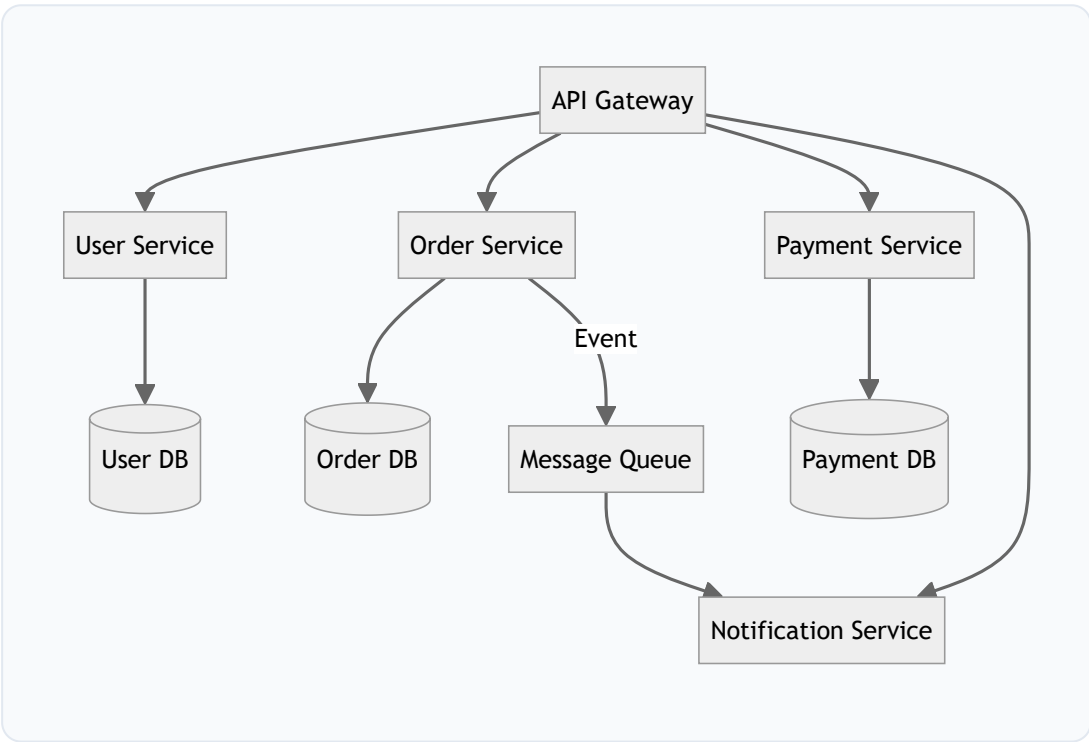


Architecture Patterns Compared

Pattern	Pros	Cons	When to Use
Monolith	Simple, easy to debug, fast to develop	Hard to scale teams and components independently	Startups, MVPs, small teams
Microservices	Independent deploy, scale, and team ownership	Network complexity, distributed debugging	Large teams, distinct domains

Pattern	Pros	Cons	When to Use
Serverless	Zero ops, auto-scaling, pay per use	Cold starts, vendor lock-in, debugging	Event-driven, variable traffic, APIs

Microservices Overview



Concurrency & Event Loops

Node.js, Python asyncio, and Go goroutines all handle concurrency differently:

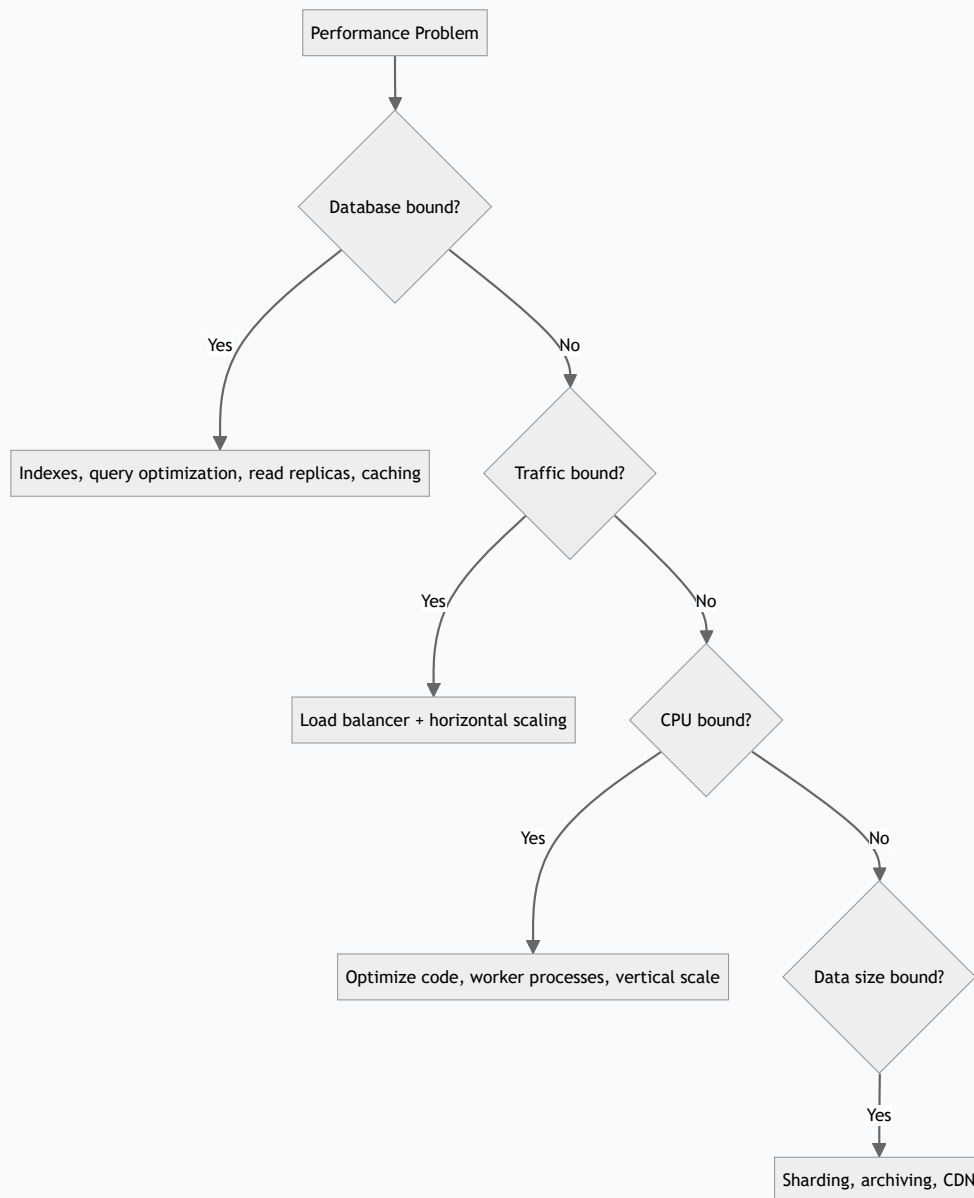
Runtime	Model	Concurrency	Best For
Node.js	Single-threaded event loop	Thousands of I/O-bound connections	APIs, real-time, I/O-heavy

Runtime	Model	Concurrency	Best For
Python asyncio	Event loop + async/await	Many concurrent I/O operations	APIs, data pipelines
Go	Goroutines (M:N threading)	Millions of lightweight threads	High- throughput services
Java/Kotlin	Thread pool	Thousands of threads	Enterprise, CPU-heavy workloads

NOTE

Async helps with I/O-bound work (database, HTTP calls). For CPU-bound work (image processing, encryption), use worker threads or separate worker processes.

Scaling Decision Framework



KEY TAKEAWAY

Build stateless, scale horizontally. Use load balancers, read replicas, and CDNs before sharding. Choose monolith vs microservices vs serverless based on team size and complexity — not hype. Understand your runtime's concurrency model and design for it.

End of Guide

Sriniously Backend Engineering Guide — Enhanced Edition

Built for engineers who want to understand the why, not just the how.